

APACHE DORIS 3.0.8

学习笔记

第三章：数据导入、实时写入与一致性

从一批字节如何变成可见 Rowset 出发，串起 Stream Load、Broker Load、Routine Load、Group Commit、Label、2PC、更新写入和故障排查。

配套答案：notes-03-a

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

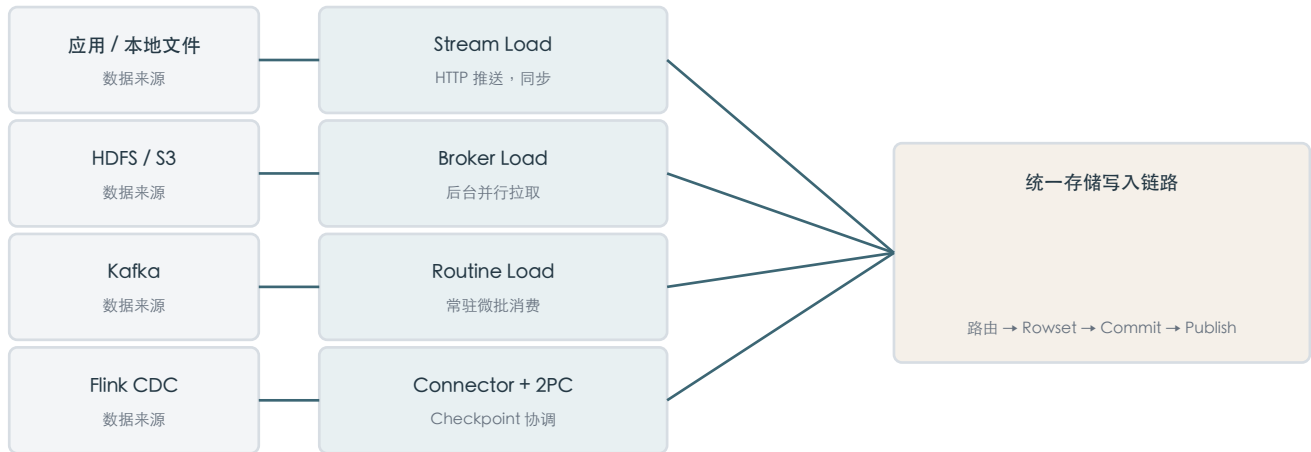
本章学习目标	4
完成本章后应能回答	4
3.1 数据导入方式全景	5
三个选择维度	5
主要方式对比	5
统一的下游链路	5
3.2 Stream Load：同步 HTTP 微批	6
FE 为什么只指路	6
物理写入路径	6
标准请求	6
一次请求的九个动作	6
响应状态的正确解释	7
3.3 Broker Load：异步批量拉取	8
为什么适合历史回灌	8
Broker 是历史名称	8
示例骨架	8
并行度的两端陷阱	8
3.4 Routine Load：Kafka 常驻微批	9
Job、Task、Transaction	9
实际并发	9
微批结束条件	9
Exactly Once 的核心	9
示例骨架	9
状态与监控	10
3.5 Group Commit 与微批设计	11
小批写入的放大链	11
Group Commit 不是新入口	11
三种模式	11
提交阈值与版本压力	11
重要限制	11
3.6 Label、2PC 与 Exactly Once	12
三种投递语义	12
三个一致性层次	12
稳定 Label	12
Stream Load 2PC	12
为什么 Unique Key 不等于 EOS	13
3.7 更新写入：UPSERT、部分列、Sequence 与删除	13

表模型先决定同 Key 语义	13
Merge-on-Write 不是原地修改	13
整行 UPSERT 与部分列更新	13
Sequence 抵抗乱序	14
CDC 删除	14
3.8 数据质量与故障排查	14
Filtered 与 Unselected	14
常见问题矩阵	15
统一排查顺序	15
成功也要做正确性审计	15
第三章综合决策图	15
一句话总复述	16
上线前检查清单	16
第三章复习题	16
自测方法	16
术语速查	17
参考资料	17

本章学习目标

本章不把导入方式当成命令集合，而是把它们放进一条统一的物理链路：数据被解析、转换、路由到 Tablet，多副本写出不可变 Rowset，事务提交后再发布为查询可见版本。

数据源不同，入口不同；进入存储层后重新汇合



完成本章后应能回答

- 为什么 Stream Load 适合应用微批，而 Broker Load 适合远程大文件。
- 为什么 Routine Load 是一个长期 Job，却由一系列短事务组成。
- 为什么高频小写会制造版本和 Compaction 压力，Group Commit 如何缓解。
- Label、事务原子性、2PC 和 Exactly Once 分别解决哪一层问题。
- Unique Key 的整行 UPSERT、部分列更新、Sequence 和删除标记怎样协作。
- 导入失败时如何从入口、数据质量、路由副本、事务发布和 Kafka 五层定位。

3.1 数据导入方式全景

Doris 导入不是把文件复制进服务器，而是把输入转换成新的 Rowset，并通过分布式事务将 Rowset 发布为可见版本。

三个选择维度

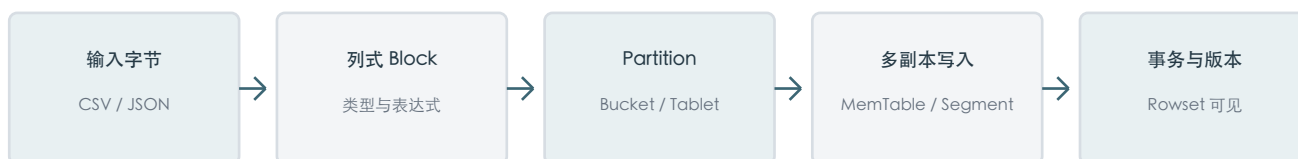
维度	问题	典型答案
数据位置	数据现在在哪里	应用内存、本地文件、Kafka、HDFS、S3、外部表
驱动方向	谁主动搬运	Push、Doris Pull、外部 Connector
生命周期	一次性还是持续	同步请求、异步有限 Job、常驻 Job
进度归属	谁保存消费位置	客户端 Label、FE Job、Kafka Offset、Flink Checkpoint

主要方式对比

方式	数据源	执行模式	最适合
Stream Load	应用、本地文件	HTTP Push，同步	秒级微批与单批结果确认
Broker Load	HDFS、S3	Doris Pull，异步	历史回灌与大文件
Routine Load	Kafka	常驻 Pull，微批事务	Kafka 到 Doris 的持续同步
INSERT VALUES	JDBC / SQL	同步	少量写入；高频时结合 Group Commit
INSERT SELECT	外部表 / Doris 表	SQL	SQL 加工后落库
Flink Connector	CDC / 流计算	Checkpoint 驱动	复杂状态计算和端到端 EOS

统一的下游链路

入口不同，但都会进入相似的存储与事务链路



选择导入方式时先问数据在哪里、是否连续、谁维护进度、允许多大可见延迟，以及失败后由谁重试。不要先从命令名字出发。

3.2 Stream Load : 同步 HTTP 微批

客户端通过 HTTP 推送一个批次；Doris 用一个事务将被接受的数据路由到目标 Tablet，并同步返回处理结果。

FE 为什么只指路

控制面走 FE，主体数据直接进入 BE



若主体数据全部经过 FE，FE 的网络、CPU 和内存会干扰元数据与调度职责。因此 FE 返回重定向，curl 使用 `--location-trusted` 跟随并携带认证。客户端还必须能访问重定向后的 BE 地址。

物理写入路径

Stream 表示边接收边处理，不表示完全没有缓冲



标准请求

```

curl --location-trusted \
  -u root:password \
  -H "Expect:100-continue" \
  -H "label:orders_20260816_001" \
  -H "format:json" \
  -H "read_json_by_line:true" \
  -H "strict_mode:true" \
  -H "max_filter_ratio:0" \
  -T orders.json \
  http://fe1:8030/api/rt_dw/ods_order/_stream_load
  
```

一次请求的九个动作

1. FE 校验入口并选择协调 BE。
2. 协调 BE 向 FE 开启事务，Label 用于识别该业务批次。
3. Reader 流式解析 CSV、JSON、Parquet 或 ORC。
4. 执行列映射、类型转换、表达式和质量检查。
5. 按 Partition Key 和 Bucket Key 路由每一行。

- 6. 目标 BE 写 MemTable，Flush 为 Segment 并组成 Pending Rowset。
- 7. 达到多数副本成功后，FE 将事务置为 COMMITTED。
- 8. Publish Version 完成后成为 VISIBLE。
- 9. 客户端解析 JSON Status 和行数，而不能只看 HTTP 200。

响应状态的正确解释

Status	真实含义	动作
Success	导入完成	核对 Loaded、Filtered、Unselected
Publish Timeout	通常已 COMMITTED，发布未及时完成	查询原事务，不换 Label 重发
Label Already Exists	同名批次已存在	检查 ExistingJobStatus 或事务状态
Fail	未成功提交	查看 Message 与 ErrorURL

原子性表示被接受的行不会按 Tablet 部分提交；如果允许脏数据过滤，错误行可被排除，剩余正确行作为整体事务提交。

3.3 Broker Load：异步批量拉取

客户端只提交 LOAD LABEL；FE 保存有限后台 Job，并让多个 BE 直接并行读取 HDFS、S3 等远程文件。

客户端断开不影响已经提交的后台 Job



为什么适合历史回灌

- 远程存储本身是耐久、可重放的数据源，不需要客户端二次中转。
- FE 按文件、大小与 BE 资源生成 Scan Range，充分利用集群并行。
- 作业异步运行，客户端不必为数百 GB 数据保持长连接。
- Label 记录整个有限 Job，可通过 SHOW LOAD 查询和受控重试。

Broker 是历史名称

Doris 3.x 的 HDFS 与 S3 常用访问能力已内置到 BE，可使用 WITH HDFS 或 WITH S3；自定义协议才可能需要外部 Broker 进程。

示例骨架

```
LOAD LABEL rt_dw.orders_backfill_20260815
(
  DATA INFILE("hdfs://namenode:8020/orders/dt=2026-08-15/*.parquet")
  INTO TABLE ods_order
  FORMAT AS "parquet"
  (order_id, user_id, amount, event_time)
  SET(event_date = to_date(event_time))
)
WITH HDFS("hadoop.username" = "doris")
PROPERTIES("timeout"="14400", "max_filter_ratio"="0");
```

并行度的两端陷阱

文件形态	问题	改进
一个巨大不可切分压缩文件	少数 Scanner 顺序读取	拆分文件或采用可并行读取格式
海量极小文件	枚举、打开和调度开销过大	先合并成合理大小文件
一个超大事务跨很多分区	失败面、内存和重试代价大	按日期或业务批次拆 Label

3.4 Routine Load : Kafka 常驻微批

Routine Load 是长期 Job，不是无限长事务；Job 持续生成短 Task，每个 Task 消费一段 Offset 并对应一个独立事务。

Job、Task、Transaction

无限流通过连续的有限事务获得可见性和故障恢复能力



实际并发

```

actual_task_num = min(
    kafka_topic_partition_num,
    desired_concurrent_number,
    doris_system_concurrency_limit
)
  
```

Topic 只有三个 Partition 时，把 desired_concurrent_number 设置为十也不会产生十路有效消费。Kafka Partition 决定上游读取并行度，Doris Tablet 决定下游存储并行度，二者不要求一一对应。

微批结束条件

阈值	作用	主要权衡
max_batch_interval	低流量时按时间提交	更短延迟 vs 更多事务
max_batch_rows	控制每批最大行数	行开销与版本开销
max_batch_size	控制每批字节数和内存	吞吐 vs 单批资源

Exactly Once 的核心

失败则事务不可见且 Offset 不推进；成功则数据和消费进度共同前进



示例骨架

```
CREATE ROUTINE LOAD rt_dw.rl_order_event ON ods_order_event
COLUMNS(order_id,user_id,amount,event_time,event_date=to_date(event_time))
PROPERTIES(
  "format"="json",
  "desired_concurrent_number"="3",
  "max_batch_interval"="10",
  "strict_mode"="true"
) FROM KAFKA(
  "kafka_broker_list"="kafka1:9092,kafka2:9092",
  "kafka_topic"="order_event",
  "property.kafka_default_offsets"="OFFSET_BEGINNING"
);
```

状态与监控

SHOW ROUTINE LOAD 应同时检查 State、CurrentTaskNum、Progress、ReasonOfStateChanged、ErrorLogUrls 和 OtherMsg。RUNNING 只表示任务活着，不表示没有 Offset 积压。PAUSE 可恢复，STOP 则结束 Job。

3.5 Group Commit 与微批设计

Group Commit 解决高频小写的固定开销：多个请求进入服务端队列，达到时间或数据量阈值后共享一个事务、版本和更大的 Rowset。

小批写入的放大链

问题不一定是数据量大，而是固定成本重复过多



Group Commit 不是新入口

它是 INSERT INTO VALUES 与 Stream Load 的服务端优化。客户端能攒批时仍应优先攒批，因为 Group Commit 不会消除连接、网络 and 每个请求的全部解析成本。JDBC 还应配合 PreparedStatement。

三种模式

模式	返回时机	可见性	适合
off_mode	独立事务完成	按普通导入	Label、2PC、部分更新或大批
sync_mode	合并事务提交后	返回后可查询	高并发且要求立即可见
async_mode	本地 WAL 持久化后	稍后才可查询	写延迟敏感、允许秒级可见

```
-- JDBC / INSERT
SET group_commit = sync_mode;
INSERT INTO event_log VALUES (1001, 'login');
```

```
# Stream Load
-H "group_commit:async_mode"
```

提交阈值与版本压力

Doris 3.0 文档中的典型默认提交窗口为 10 秒或 64 MB，任一满足即提交；实际以 3.0.8 表属性为准。缩短窗口降低可见延迟但增加版本和 Compaction，增大数据阈值提升批量效率但占用更多内存。

```
ALTER TABLE event_log SET (
  "group_commit_interval_ms" = "2000",
  "group_commit_data_bytes" = "134217728"
);
```

重要限制

- 指定用户 Label 的请求会退化为普通导入，因为独立身份与共享事务冲突。
- Stream Load 2PC 会退化为独立事务，外部协调者不能控制含有其他请求的共享事务。
- 部分列更新、显式事务和某些表达式写入也可能不进入 Group Commit。

- Unique Key 的提交顺序不保证等同于请求发起顺序，乱序更新必须使用 Sequence。
- async_mode 先确认本地单副本 WAL；需监控 WAL 磁盘并规范 Decommission。

3.6 Label、2PC 与 Exactly Once

Exactly Once 不是网络只传一次，而是无论重试多少次，最终业务效果只发生一次。它要求源端进度与 Doris 事务建立可恢复的对应关系。

三种投递语义

语义	失败策略	可能结果
At Most Once	失败不重试	不重复，但可能丢
At Least Once	超时就重试	尽量不丢，但可能重复
Exactly Once	允许重试且识别旧批次	传输可多次，逻辑效果一次

三个一致性层次

原子性、幂等和端到端 EOS 不能互相替代



稳定 Label

```

source      = order_event
partition   = 3
start_offset = 1000
end_offset   = 2000
label       = order_event_p3_1000_2000
  
```

相同业务批次必须重新计算出同一个 Label，批次内容也必须稳定。网络超时后先查询原 Label，不能生成随机重试 Label。Label 会按时间与数量清理，超过去重窗口后同名 Label 可能再次成功，因此长期状态还应由外部系统保存。

Stream Load 2PC

两阶段把写入完成与最终提交决定分开



```
#
-H "label:flink_checkpoint_100"
-H "two_phase_commit:true"

#
curl -X PUT --location-trusted -u root:password \
  -H "txn_id:18036" -H "txn_operation:commit" \
  http://fe1:8030/api/rt_dw/ods_order/_stream_load_2pc
```

为什么 Unique Key 不等于 EOS

Unique Key 可能让两次相同主键写入最终只显示一行，但 Aggregate Key 会重复累加，Duplicate Key 会保留两条，乱序覆盖与 Rowset 开销仍然存在。因此它是结果层幂等防线，不是传输协议。Group Commit 多请求共享事务，也不能直接提供逐请求 Exactly Once。

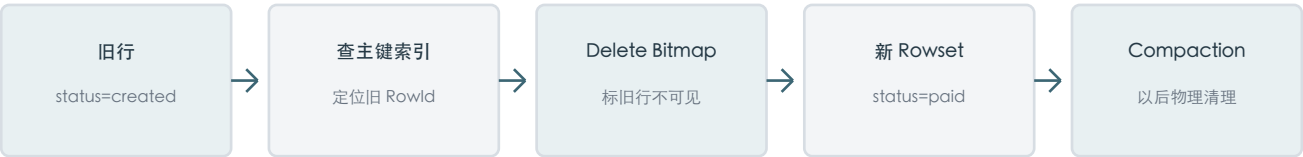
3.7 更新写入：UPSERT、部分列、Sequence 与删除

表模型先决定同 Key 语义

模型	同 Key 写入	典型用途
Duplicate Key	全部追加保留	事件明细、日志、审计
Unique Key MOW	新行覆盖旧行的逻辑效果	订单当前态、画像、CDC
Aggregate Key	按 SUM、MAX、REPLACE 等合并	实时指标与预聚合

Merge-on-Write 不是原地修改

不可变 Segment 通过追加新版本与失效旧行实现更新



整行 UPSERT 与部分列更新

只在 INSERT 中列出少数列，不等于自动部分更新；未开启 partial update 时，缺失列会被 NULL 或默认值补齐并覆盖旧值。

```
# Stream Load
-H "partial_columns:true"
-H "columns:order_id,status"

-- INSERT
SET enable_unique_key_partial_update = true;
INSERT INTO order_current(order_id,status) VALUES (1,'paid');
```

3.0.8 要求一次部分更新批次中的每一行更新相同列集合，且必须包含全部 Key 列。每行可更新不同列的 Flexible Partial Update 从 3.1.0 才支持。部分更新需要读取旧行补齐缺失列并写出完整新行，会产生读写放大；宽表高频更新可评估高速 SSD 与 store_row_column。

Sequence 抵抗乱序

```
PROPERTIES (
  "enable_unique_key_merge_on_write" = "true",
  "function_column.sequence_col" = "update_time"
)
```

相同 Key 下，Sequence 大的替换小的，旧事件晚到不能让订单状态倒退。Sequence 应来自业务版本、binlog 位点或可靠更新时间；尽量避免相同值，也不要再用 Doris 接收时间替代业务顺序。

CDC 删除

```
CREATE ROUTINE LOAD rt_dw.rl_order_cdc ON order_current
WITH MERGE
COLUMNS(order_id,user_id,amount,status,update_time,op)
DELETE ON op = 'D'
ORDER BY update_time
PROPERTIES("format"="json")
FROM KAFKA(...);
```

删除会写入 __DORIS_DELETE_SIGN__ 逻辑标记，查询立即忽略该主键，磁盘空间由后续 Compaction 物理回收。小范围临时修正可使用 UPDATE/DELETE SQL；已知主键的大规模实时变化更适合 Load UPSERT。

3.8 数据质量与故障排查

排查时先定位失败层级，再看对应证据：入口、解析质量、分区与副本、事务发布、Kafka Job。不要一上来只调大容错率或反复重试。

五层排查路径



Filtered 与 Unselected

$$\text{filter_ratio} = \text{FilteredRows} / (\text{FilteredRows} + \text{LoadedRows})$$

```
FilteredRows :
UnselectedRows : WHERE /
```

strict_mode 决定某些转换失败是否判为错误行，max_filter_ratio 决定整批最多容忍多少错误行。把 max_filter_ratio 调到 1 可能让任务 Success 却加载 0 行，必须同时核对行数与 ErrorURL。

常见问题矩阵

现象	常见根因	正确方向
307 / 连接 BE 失败	未跟随重定向或网络不通	检查 --location-trusted 与 BE 可达性
too many filtered rows	格式、类型、长度、映射错误	查看 ErrorURL，不盲目放宽比例
no partition for tuple	分区未创建、日期或时区错误	补分区并核对路由字段
failed to meet quorum	BE、磁盘或副本不健康	恢复多数派与 Tablet 健康
too many versions	高频小批且 Compaction 落后	攒批、Group Commit、检查 IO
Publish Timeout	事务已提交但发布延迟	SHOW TRANSACTION，不新 Label 重发
Routine Load PAUSED	质量、Kafka、Offset 或表异常	看 Reason、ErrorLogUrls 后再恢复
Offset out of range	Kafka retention 已清理旧位置	决定补数或重置，不能静默跳过

统一排查顺序

1. 记录导入方式、Label、TxnId、请求时间和源端批次。
2. 检查 Status 或 Job State，不能只看 HTTP 200 或 RUNNING。
3. 核对 Total、Loaded、Filtered、Unselected，并读取 ErrorURL。
4. 确认表模型、列映射、整行或部分列模式、Sequence 与 Delete 语义。
5. 检查分区、桶路由、热点、Tablet 副本、BE 磁盘和内存。
6. 用 SHOW TRANSACTION 区分 PREPARE、COMMITTED、VISIBLE、ABORTED。
7. 持续任务再对齐 Kafka 最新 Offset、Routine Load Progress 与业务时间。

成功也要做正确性审计

- TotalRows 应能与 Loaded + Filtered + Unselected 对账。
- Unique Key 的 LoadedRows 不等于表行数增加量，因为其中可能是更新。
- 抽样检查主键最终状态、Sequence、删除结果、时间范围与分区归属。
- 长期记录 source_batch_id、Label、TxnId、Offset 区间和全部行数指标。

第三章综合决策图

需求	推荐入口	一致性抓手	关键风险
应用实时微批	Stream Load	稳定 Label	批次太小、重试换 Label
远程历史回灌	Broker Load	Job Label	单事务过大、文件形态差
Kafka 直接同步	Routine Load	Offset + 事务	积压、retention、脏数据暂停
Flink CDC	Connector + 2PC	Checkpoint + TxnId	乱序、恢复点与版本兼容
微服务高频小写	Group Commit	按业务选择结果幂等	async WAL、逐请求无 Label
订单当前态	Unique Key MOW	Sequence + Delete Sign	部分更新放大、旧事件晚到
事件明细	Duplicate Key	事件 ID / 外部去重	重复投递会真实保留

一句话总复述

入口负责把数据送到 Doris；表模型决定同 Key 的业务语义；事务保证一个批次的原子性；Label 负责批次幂等；2PC 或 Offset 协调提供端到端一致性；Rowset、版本和 Compaction 决定持续写入能否长期稳定。

上线前检查清单

- 明确源端位置、批次边界、可见延迟和失败责任人。
- 选择 Duplicate、Unique 或 Aggregate，明确是否需要 Sequence 与删除语义。
- 显式设置稳定 Label 或使用 Routine Load / Connector 的一致性机制。
- 控制单批大小、并发、涉及分区数，避免小 Rowset 风暴。
- 将行数、ErrorURL、事务状态、Offset Lag、WAL 和 Compaction 纳入监控。
- 设计回放与补数流程；Exactly Once 不能挽救已被 Kafka retention 删除的数据。

第三章复习题

1. 为什么所有导入方式最终都可以用 Rowset、事务与 Publish 解释？
2. Stream Load 为什么由 FE 重定向到 BE？客户端网络需要满足什么条件？
3. Publish Timeout 与普通失败有什么本质区别？
4. 为什么 Broker Load 适合大规模历史回灌？文件过大或过小各有什么问题？
5. Routine Load Job、Task 和 Transaction 的生命周期分别是什么？
6. Routine Load 如何让 Kafka Offset 与 Doris 数据不丢不重？
7. 高频小批为什么会同时伤害 FE、Tablet 版本和 Compaction？
8. Group Commit sync_mode 与 async_mode 成功返回时分别意味着什么？
9. 为什么用户 Label、Stream Load 2PC 与真正的 Group Commit 存在冲突？
10. 事务原子性、Label 幂等和端到端 Exactly Once 分别解决什么？
11. Unique Key 为什么不能替代 Exactly Once 协议？
12. 整行 UPSERT 与部分列更新有什么危险差异？
13. 部分列更新为什么会产生读写放大，3.0.8 有什么批次限制？
14. Sequence Column 怎样阻止旧 CDC 事件覆盖新状态？
15. 遇到 too many filtered rows、Publish Timeout 和 Offset out of range 应分别怎样处理？

自测方法

每题先用一句话给结论，再画出因果链，最后指出一个边界或常见误区。答案请使用独立的 notes-03-a，避免边看边背。

术语速查

术语	一句话含义
Label	数据库范围内标识一个事务或导入批次的幂等身份
Pending Rowset	物理已写出但尚未发布为查询可见的 Rowset
COMMITTED	事务可靠提交，但版本可能尚未发布
VISIBLE	版本发布完成，之后启动的查询可以读取
WAL	async Group Commit 提前确认前写入的本地预写日志
Sequence	相同主键下决定业务新旧顺序的比较值
Delete Sign	CDC 删除的逻辑标记，后续由 Compaction 物理清理

参考资料

以下链接用于核对 Doris 3.0.8 相关机制；功能边界以 3.0 文档为准，3.x 页面用于补充稳定原理。

- [1] Doris 3.0 Stream Load
<https://doris.apache.org/docs/3.0/data-operate/import/import-way/stream-load-manual/>
- [2] Doris 3.0 Broker Load
<https://doris.apache.org/docs/3.0/data-operate/import/import-way/broker-load-manual/>
- [3] Doris 3.0 Routine Load
<https://doris.apache.org/docs/3.0/data-operate/import/import-way/routine-load-manual/>
- [4] Doris 3.0 Group Commit
<https://doris.apache.org/zh-CN/docs/3.0/data-operate/import/group-commit-manual/>
- [5] Doris 3.0 Transaction
<https://doris.apache.org/docs/3.0/data-operate/transaction/>
- [6] SHOW TRANSACTION
<https://doris.apache.org/docs/3.0/sql-manual/sql-statements/transaction/SHOW-TRANSACTION>
- [7] Partial Column Update
<https://doris.apache.org/docs/3.0/data-operate/update/partial-column-update/>
- [8] Unique Update Concurrent Control
<https://doris.apache.org/docs/3.0/data-operate/update/unique-update-concurrent-control/>
- [9] Handling Messy Data
<https://doris.apache.org/docs/3.x/data-operate/import/handling-messy-data/>
- [10] Stream Load Principle
<https://doris.apache.org/blog/principle-of-Doris-Stream-Load/>
- [11] Flink Real-time Write
<https://doris.apache.org/blog/Flink-realtime-write/>
- [12] Release 3.0.8
<https://doris.apache.org/releases/v3.0/release-3.0.8/>